OmniCart

A Relational E-Commerce Database Management System with AI-Powered Product Recommendations

PROJECT PROPOSAL

Project Title	***OmniCart***: A Relational E-Commerce Database Management System with AI-Powered Product Recommendations
Course	CS 2005: Database Systems
Instructor	Sir Shoaib Rauf
Submitted By	Rayyan Aamir | 24K-0687
Group Members	Rayyan Aamir ***(Group Leader)*** | 24K-0687 Ahmed Shah Rashdi | 24K-0709 Muhammad Usaid Khan | 24K-0832
Submission Date	11th March, 2026

1. Introduction and Brief Description of the System

OmniCart is a full-stack, web-based E-Commerce Database Management System designed to manage the complete lifecycle of an online retail platform. The system provides a structured relational database backend built on MySQL that handles all core e-commerce operations including product cataloging, order processing, customer management, inventory tracking, and seller administration.

What distinguishes ***OmniCart*** from a conventional e-commerce system is the seamless integration of Artificial Intelligence into the database-driven platform. The system leverages Machine Learning algorithms trained on the Amazon Product Reviews Dataset to deliver personalized product recommendations to each customer. An integrated OpenAI GPT-4 API layer generates natural language explanations for every recommendation, making the shopping experience intuitive and transparent. Additionally, an n8n automation layer handles all triggered communications such as order confirmations, cart abandonment reminders, and weekly personalized deal digests.

The system is designed around three distinct user roles - Customer, Seller, and Administrator - each interacting with the database through a dedicated web-based portal. All database interactions are managed through a Python Flask backend with SQLAlchemy ORM, ensuring clean separation between the business logic, AI layer, and data layer.

2. System Functionality

OmniCart performs the following core functions across all user roles:

2.1 Customer Functions

- User Registration and Authentication: Customers can register, log in, and manage their profiles securely using JWT-based authentication.
- Product Browsing and Search: Customers can browse products by category, search by keyword, and filter by price, brand, and rating.
- AI-Powered Recommendations: The system automatically generates personalized product recommendations based on the customer's purchase history and browsing behavior using collaborative filtering.
- Shopping Cart Management: Customers can add, update, and remove items from their cart. Cart data is persisted in the database.
- Order Placement with Transaction: Placing an order triggers an atomic SQL transaction that inserts the order, deducts stock, and clears the cart simultaneously.

- Review Submission: Customers can submit star ratings and written reviews. The system automatically runs sentiment analysis on review text and stores the sentiment score in the database.
- Order History: Customers can view their full order history with product details, order status, and invoice summaries.

2.2 Seller Functions

- Product Management: Sellers can add new products, update prices and descriptions, manage stock levels, and delete listings.
- AI Description Generator: When adding a product, the seller can request an AI-generated product description using the OpenAI GPT-4 API.
- Sales Analytics: Sellers can view sales reports showing total revenue, best-selling products, and low-stock alerts.
- Order Fulfillment Tracking: Sellers can view and update the status of orders containing their products.

2.3 Administrator Functions

- Full CRUD Operations: Administrators have complete Create, Read, Update, and Delete access over all tables including Users, Products, Categories, Orders, and Reviews.
- User Management: Admins can activate, deactivate, or delete customer and seller accounts.
- Platform Analytics Dashboard: Admins can view platform-wide metrics including top-selling products, most active customers, average sentiment scores by category, and revenue trends.
- Category Management: Admins manage the product category hierarchy stored in the Categories table.
- Recommendation Audit: Admins can view the Recommendations table to audit which products were recommended to which users and with what confidence score.

2.4 Automated System Functions

- Order Confirmation Automation: Immediately after an order is placed, n8n triggers an automated confirmation email to the customer containing order details.
- Cart Abandonment Reminder: n8n checks the Cart table every 24 hours and sends reminder emails to customers with items still in their cart.
- Weekly Deals Digest: Every Monday, n8n queries the Recommendations table and sends each customer a personalized email listing their top 5 recommended products.
- Low Stock Alert: n8n monitors the Products table and automatically alerts sellers when stock falls below 10 units.

3. Users of the System

The system defines three user roles, each with distinct permissions and database access levels:

User Role	Description	Key Capabilities
Customer	End-users who browse, purchase, and review products on the platform.	Browse products, manage cart, place orders, write reviews, view AI recommendations, track order history
Seller	Registered vendors who list and sell products on the platform.	Add/edit/delete products, view sales analytics, manage inventory, use AI description generator, track order fulfillment
Administrator	Platform managers with full system access and oversight.	Full CRUD on all tables, user account management, category management, platform analytics, recommendation auditing

4. Technologies Used

4.1 Front-End Technologies

Technology	Purpose
React.js	Component-based single-page application framework for building dynamic, reactive UI and role-based dashboards
Tailwind CSS	CSS framework for responsive layout, styling, and consistent UI design across all pages
Chart.js	Rendering analytics charts and sales graphs on admin and seller dashboards

4.2 Back-End Technologies

Technology	Purpose
Python 3.11	Primary backend programming language
Flask	Lightweight web framework for REST API development
SQLAlchemy	ORM for database interaction and model definition
Flask-JWT-Extended	JWT-based authentication and role-based access control
OpenAI API (GPT-4)	AI-generated product descriptions and recommendation explanations
Scikit-learn	Collaborative filtering model (SVD) and cosine similarity search
NLTK / TextBlob	Natural language processing for review sentiment analysis
n8n	Workflow automation for emails, alerts, and scheduled tasks

4.3 Database Technologies

Technology	Purpose
MySQL 8.0	Primary relational database management system
MySQL Workbench	ERD design, query execution, and database administration
SQL	Mandatory language for all database operations — DDL, DML, Views, Transactions

5. Scope of the System

5.1 What the System WILL Cover

- A fully functional web-based e-commerce platform accessible through a browser.
- A relational MySQL database with 8 normalized tables covering all core entities.
- Complete user registration, authentication, and role-based access for Customer, Seller, and Admin.
- Full product lifecycle management: listing, browsing, purchasing, and reviewing products.
- An AI-powered product recommendation engine using Collaborative Filtering (SVD matrix factorization) trained on the Amazon Product Reviews Dataset.
- Sentiment analysis on customer reviews using NLP, with sentiment scores stored in the Reviews table.
- Natural language recommendation explanations generated via OpenAI GPT-4 API.
- Atomic transaction handling for order placement ensuring data consistency.
- Database Views for customer order history and top-rated products reporting.
- SQL Joins, aggregate functions, built-in SQL functions, and dynamic user input handling.
- n8n automation workflows for order confirmations, cart reminders, weekly digests, and stock alerts.
- An admin analytics dashboard with platform-wide metrics powered by advanced SQL queries.

5.2 What the System WILL NOT Cover

- Real payment gateway integration (e.g., Stripe, PayPal) — payments will be simulated.
- Mobile application development — the system is web-browser based only.
- Real-time chat or customer support messaging features.
- Shipping and logistics tracking integration with third-party courier APIs.
- Multi-language or multi-currency support in this version.
- Social media login (OAuth) — standard email/password authentication only.

6. Queries Per User Role

As required, at least one query will be implemented for each user role. All queries will make use of JOINs, built-in SQL functions, and dynamic user input handling.

6.1 Customer Queries

- View full order history: JOIN across Users, Orders, Order_Items, and Products tables filtered by logged-in customer ID.
- Search products by keyword, category, and price range using dynamic input with LIKE, WHERE, and JOIN on Categories.
- View personalized recommendations: SELECT from Recommendations JOIN Products WHERE user_id = logged-in user.

6.2 Seller Queries

- View sales summary: SUM, COUNT aggregate functions on Order_Items JOIN Products JOIN Orders filtered by seller_id.
- Low stock report: SELECT products WHERE stock < 10 AND seller_id = current seller with ORDER BY stock ASC.
- Product performance: AVG(rating), COUNT(reviews) from Reviews JOIN Products WHERE seller_id = current seller.

6.3 Administrator Queries

- Platform revenue report: SUM of total_amount from Orders with GROUP BY month using DATE_FORMAT built-in function.
- Top-rated products view: Uses the TopRatedProducts VIEW with AVG rating, total review count per product.
- User activity report: COUNT of orders per customer using JOIN across Users and Orders with HAVING and ORDER BY.

7. Assumptions

- Each product belongs to exactly one seller (user with role = seller).
- A customer can place multiple orders, and each order can contain multiple products via the Order_Items junction table.
- Sentiment analysis scores range from 0.0 (most negative) to 1.0 (most positive) and are computed automatically upon review submission.
- The AI recommendation engine will be pre-trained on the Amazon Product Reviews Dataset (Kaggle) before system deployment. The trained model will be loaded at runtime.
- n8n will be hosted locally during development and connected to the Flask backend via HTTP webhooks.
- Product prices are stored in USD (US Dollars) and no currency conversion is performed.
- All users must be registered and authenticated to perform any transactional operation (cart, orders, reviews).

8. Preliminary Entity Overview

The database will consist of approximately 8-9 relational tables across 6 core entities, satisfying the project requirement. The following table summarizes the planned entities and their primary attributes:

Entity / Table	Primary Key	Key Attributes	Relationships
customers	customer_id	name, email, phone, address, dob	Orders, Reviews, Cart, Recommendations
vendors	vendor_id	name, email, store_name, is_approved	Products, Orders
categories	category_id	name, description, parent_category_id	Products
products	product_id	vendor_id, category_id, name, price, stock_qty	Categories, Vendors, Order Items, Reviews
orders	order_id	customer_id, status, total_amount, placed_at	Customers, Order Items, Payments
order_items	item_id	order_id, product_id, quantity, unit_price	Orders, Products
payments	payment_id	order_id, method, status, amount, paid_at	Orders

Entity / Table	Primary Key	Key Attributes	Relationships
reviews	review_id	customer_id, product_id, rating, comment	Customers, Products
recommendations	rec_id	customer_id, product_id, score, generated_at	Customers, Products

A full Entity-Relationship Diagram (ERD) will be submitted with the final project report. As required by the instructor, two versions will be provided: (1) a hand-drawn diagram on paper, and (2) a software-generated version. Both will be included in the ZIP submission on Google Classroom.

9. Stored Procedures / Subprograms Per User Role

As required, at least one stored procedure (subprogram) will be implemented per user role. Each procedure encapsulates a critical business operation for that role and is invoked from the Flask back-end via a CALL statement.

9.1 Customer - PlaceOrder Procedure

Procedure Name: PlaceOrder(IN p_customer_id INT, IN p_cart_items JSON)

Purpose: Encapsulates the complete order placement transaction. The procedure (1) inserts a new record into the orders table, (2) iterates over each cart item and inserts into order_items, (3) deducts stock from the products table for each item, and (4) clears the customer's cart. All steps execute within a single atomic transaction — if any step fails, the entire transaction is rolled back to maintain data consistency.

9.2 Seller - UpdateProductInventory Procedure

Procedure Name: UpdateProductInventory(IN p_product_id INT, IN p_vendor_id INT, IN p_new_stock INT, IN p_new_price DECIMAL)

Purpose: Allows a seller to update the stock quantity and price of one of their products. The procedure first validates that the product belongs to the calling vendor (ownership check) before applying updates. If the vendor_id does not match, the procedure raises a user-defined SQL error and no update is applied. This prevents sellers from modifying another vendor's listings.

9.3 Administrator - DeactivateUser Procedure

Procedure Name: DeactivateUser(IN p_user_id INT, IN p_user_type VARCHAR(20), IN p_reason VARCHAR(255))

Purpose: Allows an administrator to deactivate a customer or seller account. The procedure sets the account status to 'inactive' in the appropriate table, cancels any pending orders associated with the account, and logs the deactivation action along with the reason in an audit log. The entire operation is wrapped in a transaction to ensure the account status change and order cancellations are applied atomically.